Donald Van Steenwyk

ISTM 622 M6 JSON Milestone Deliverable

1. Video Link: Clickable URL to your Unlisted YouTube video.

2. Honor Code & AI Attestation.
On my honor, as an Aggie, I have neither given nor received unauthorized assistance on this assignment. I further affirm that I have not and will not provide this code to any person, platform, or repository, without the express written permission of Dr. Gomillion. I understand that any violation of these standards will have serious repercussions. The only AI assistance I used was the Google search summary and Claude. I followed the links provided in the summary to verify it was not hallucination and tested any code it helped me write to ensure it worked correctly. Claude helped me understand the reflection question and some aggregation functions better so that I could put it in my own words/code. Claude also helped me diagnose a persistent issue with mongodb eating system resources.

The Pipeline Scripts: Paste the text of your User Data script, your sync.sh script, and your crontab entry.
User data script
sync.sh
crontab

```
#!/bin/bash
# TODO make things consistent and pretty
# (This will probably never happen)

#set -e

# config

# CHANGE to your UNAME and password
# TODO aws secretsmanager
UNAME="dvansteenwyk"
UPASSWORD="notthepassword"
ADMINPASS="sosecurenow"
TAMUUID="126009757"

# For discarding pasted bracketed text that comes after an error
UNPASTE='echo "FAILED\n" && printf "\e[?2004l"'

IMAGEID="ami-0ec10929233384c7f"   # Ubuntu server 24.04 LTS
INSTTYPE="t3.micro"
KEYNAME="vockey"
# CHANGE to whatever you want or a groupname you already have
SGNAME="SSHOnly"
SGDESC="A security group for allowing ssh and other galera cluster members"
# Grab Security Group ID from JSON query
```

```
SGIDPARSE="SecurityGroups[*].GroupId"
# This is only used for ssh. To make extra secure use your own IP address
MYIP="0.0.0.0/0" # use curl https://checkip.amazonaws.com if running from home
# AWS CLI command to get AccountID in plain text
MYACCT="$(aws sts get-caller-identity --query "Account" --output text)"
# Ports needed for MariaDB
PORTS=("tcp 3306")
# This iam profile will allow us to send remote commands
IAMPROFILE="LabInstanceProfile"
# Update this when a new stable version releases
MARIADBVER="11.8"

# Outfiles for JSON Milestone
OUTFILE_DIR="/var/lib/mysql_files"

PROD_OUT="${OUTFILE_DIR}/prod.json"
CUST_OUT="${OUTFILE_DIR}/cust.json"
CUSTOM1_OUT="${OUTFILE_DIR}/custom1.json"
CUSTOM2_OUT="${OUTFILE_DIR}/custom2.json"

# Cron
CRON_CMD="flock -n /tmp/mysync.lock /root/sync.sh"
CRON_JOB="*/15 * * * * $CRON_CMD"


WAIT_TIMEOUT=200 # Default of 'aws ssm wait command-executed' is 100
POLL_INTERVAL=5  # Default of 'aws ssm wait command-executed' is 5
gen_poll_wait() { # $1=POLLCOMM $2=TARGETS $3=WAITMSG
  DURATION=0
  COMMSTATUS=`${1}`
  local -n TARGETS=$2
  # Regex for 'while COMMSTATUS NOT IN TARGETS'
  while [[ -v TARGETS["$COMMSTATUS"] && \
        $DURATION -lt $WAIT_TIMEOUT ]]; do
    echo "$3"
    sleep $POLL_INTERVAL
    COMMSTATUS=`${1}`
    if [ $? -ne 0 ]; then
      $("${UNPASTE}")
    fi
    let DURATION=DURATION+POLL_INTERVAL
  done

  echo "Command finished with status: $COMMSTATUS"

  if [ $? -ne 0 ]; then
    $("${UNPASTE}")
```

```
  fi
}

# This is a more versitile version of 'aws ssm wait command-executed'
gci_poll_wait() { #$1=COMMID $2=INSTANCEID $3=WAITMSG
  TERM_STATS=( "Success" "Cancelled" "TimedOut" "Failed" )
  declare -A UNTERM_STATS
  UNTERM_STATS=( ["Pending"]=1 ["InProgress"]=1 \
                 ["Delayed"]=1 ["Cancelling"]=1 )
  POLLCOMM="aws ssm get-command-invocation \
    --command-id $1 \
    --instance-id $2 \
    --query Status \
    --output text"
  gen_poll_wait "$POLLCOMM" UNTERM_STATS "$3"
}

comm_send_wait() { #$1=INSTANCEID $2=COMMAND $3=WAITMSG $4=RETURN_COMMID
  # Use the variable passed as reference to set COMMID
  # Used for checking for return status after command terminates
  local -n CID=$4
  CID=$( aws ssm send-command \
    --instance-ids "$INSTANCEA" \
    --document-name "AWS-RunShellScript" \
    --parameters \
      "commands=[$2]" \
    --region us-east-1 \
    --query "Command.CommandId" \
    --output text )

  sleep 1
  gci_poll_wait "$CID" "$1" "$3"
}

# construct IP permissions for a single security group rule
ip_permissions_str() { #$1=SGID $2=PROTOCOL $3=PORT $4=MYUSERID
  echo "IpProtocol=$2,FromPort=$3,ToPort=$3, \
    UserIdGroupPairs=[{UserId=$4,GroupId=$1}]" \
    | tr -d "[:space:]"
}

IFS= read -r -d '' STARTUP <<EOF
#!/bin/bash
# shebang so the shell knows this is a shell script
# use tail -f /var/log/user-data.log | grep "mariadb" to check mariadb status
# use tail -f /var/log/user-data.log | grep -m 1 "SUCCESS" if you are confident
```

```
set -e # Exit script if error
set -u # Unset variable references cause error -> exit
set -o # Catch pesky |bugs| hiding in pipes

touch /root/0-script-failed

# Log all stdout output to this log file
exec > >(tee /var/log/user-data.log|logger -t user-data -s 2>/dev/console) 2>&1
# Politely ask tools not to attempt to run interactively
export DEBIAN_FRONTEND=noninteractive

touch /root/1-script-started

# Standard update package lists and then packages
apt-get update
apt-get upgrade -y

touch /root/2-packages-upgraded

# Use curl to grab files from MariaDB website
apt-get install -y apt-transport-https curl

# Install MariaDB ${MARIADBVER}
# Configure the official gpg key from Maria DB Foundation
mkdir -p /etc/apt/keyrings
curl -o /etc/apt/keyrings/mariadb-keyring.pgp \
'https://mariadb.org/mariadb_release_signing_key.pgp'
# Add MariaDB Repository and Key Repository
cat > /etc/apt/sources.list.d/mariadb.sources <<SUBEOF
X-Repolib-Name: MariaDB
Types: deb
URIs: https://mirrors.accretive-networks.net/mariadb/repo/${MARIADBVER}/ubuntu
Suites: noble
Components: main main/debug
Signed-By: /etc/apt/keyrings/mariadb-keyring.pgp
SUBEOF

# update package list from mariadb repo
apt-get update
# install the mariadb-server and galera packages from the newly added repo
apt-get install -y mariadb-server galera-4

# MariaDB automatically enables and starts as a
# part of the Ubuntu package install process
#systemctl stop mariadb
#systemctl disable mariadb
```

```
touch /root/3-mariadb-installed

apt-get install -y gnupg
curl -fsSL https://pgp.mongodb.com/server-8.0.asc | \
   gpg -o /usr/share/keyrings/mongodb-server-8.0.gpg --dearmor

echo "deb [ arch=amd64,arm64 \
            signed-by=/usr/share/keyrings/mongodb-server-8.0.gpg ] \
    https://repo.mongodb.org/apt/ubuntu noble/mongodb-org/8.0 multiverse" \
    | tee /etc/apt/sources.list.d/mongodb-org-8.0.list

apt-get update
apt-get install -y mongodb-org

systemctl enable mongod
systemctl start mongod

touch /root/4-mongodb-installed

# TODO aws secretsmanager workflow
useradd -m -p ${UPASSWORD} -s /bin/bash ${UNAME}

touch /root/5-user-setup

apt-get install unzip -y
sudo -u ${UNAME} cat > /home/${UNAME}/etl.sql <<'SUBEOF'
DROP DATABASE IF EXISTS POS;
CREATE DATABASE POS;
USE POS;

CREATE TABLE Customer (
  id SERIAL PRIMARY KEY,
  firstName VARCHAR(32),
  lastName VARCHAR(30),
  email VARCHAR(128),
  address1 VARCHAR(100),
  address2 VARCHAR(50),
  phone VARCHAR(32),
  birthdate DATE,
  zip DECIMAL(5) ZEROFILL
) ENGINE=InnoDB; /* Don't forget to explicitly set engine */

CREATE TABLE City (
  zip DECIMAL(5) ZEROFILL PRIMARY KEY,
  city VARCHAR(32),
  \`state\` VARCHAR(4)
) ENGINE=InnoDB;
```

```
CREATE TABLE \`Order\` (
  id SERIAL PRIMARY KEY,
  datePlaced DATE,
  dateShipped DATE,
  customer_id BIGINT UNSIGNED
) ENGINE=InnoDB;

CREATE TABLE Orderline (
  order_id BIGINT UNSIGNED NOT NULL,
  product_id BIGINT UNSIGNED NOT NULL,
  quantity INTEGER,
  PRIMARY KEY (order_id, product_id)
) ENGINE=InnoDB;

CREATE TABLE Product (
  id SERIAL PRIMARY KEY,
  name VARCHAR(128),
  currentPrice DECIMAL(6,2),
  availableQuantity INTEGER
) ENGINE=InnoDB;

CREATE TABLE PriceHistory (
  id SERIAL PRIMARY KEY,
  oldPrice DECIMAL(6,2),
  newPrice DECIMAL(6,2),
  ts TIMESTAMP,
  product_id BIGINT UNSIGNED
) ENGINE=InnoDB;


/* Avoid defining FKs in CREATE because this is less painful */
ALTER TABLE Customer ADD CONSTRAINT FK_Customer_City_zip
  FOREIGN KEY (\`zip\`)
  REFERENCES \`City\`(\`zip\`);
ALTER TABLE \`Order\` ADD CONSTRAINT FK_Order_Customer_customer_id
  FOREIGN KEY (\`customer_id\`)
  REFERENCES \`Customer\`(\`id\`);
ALTER TABLE \`Orderline\` ADD CONSTRAINT FK_Orderline_Order_order_id
  FOREIGN KEY (\`order_id\`)
  REFERENCES \`Order\`(\`id\`);
ALTER TABLE \`Orderline\` ADD CONSTRAINT FK_Orderline_Product_product_id
  FOREIGN KEY (\`product_id\`)
  REFERENCES \`Product\`(\`id\`);
ALTER TABLE \`PriceHistory\` ADD CONSTRAINT FK_PriceHistory_Product_product_id
  FOREIGN KEY (\`product_id\`)
  REFERENCES \`Product\`(\`id\`);
```

```
/* Create a table for the raw data with types suited for them */
CREATE TABLE CustomerStaging (
  id BIGINT UNSIGNED PRIMARY KEY,
  firstName VARCHAR(32),
  lastName VARCHAR(30),
  city VARCHAR(32),
  \`state\` VARCHAR(4),
  zip VARCHAR(10),
  address1 VARCHAR(100),
  address2 VARCHAR(50),
  email VARCHAR(128),
  birthdate VARCHAR(20)
);

LOAD DATA LOCAL INFILE '/home/${UNAME}/customers.csv'
  INTO TABLE CustomerStaging FIELDS TERMINATED BY ','
    OPTIONALLY ENCLOSED BY '"'
  IGNORE 1 LINES;
/* Use a select to transform the data as it is more readable
   than load data substitutions */
INSERT INTO City (
  SELECT CAST(zip AS DECIMAL(5)),
      TRIM('"' FROM city),
      \`state\`
    FROM CustomerStaging
    GROUP BY zip
);
INSERT INTO Customer (
  SELECT id,
      TRIM('"' FROM firstName),
      TRIM('"' FROM lastName),
      NULLIF(email, ''),
      TRIM('"' FROM address1),
      NULLIF(TRIM('"' FROM address2), ''),
      NULL,
      STR_TO_DATE(birthdate, '%m/%d/%Y'),
      CAST(zip AS DECIMAL(5))
    FROM CustomerStaging
);

DROP TABLE CustomerStaging; /* Done with it */

/* Create a table for the raw data with types suited for them */
CREATE TABLE OrderStaging (
  id BIGINT UNSIGNED,
  customer_id BIGINT UNSIGNED,
```

```
  datePlaced VARCHAR(30),
  dateShipped VARCHAR(30)
);
LOAD DATA LOCAL INFILE '/home/${UNAME}/orders.csv'
  INTO TABLE OrderStaging FIELDS TERMINATED BY ','
    OPTIONALLY ENCLOSED BY '"'
  IGNORE 1 LINES
;
/* Use a select to transform the data as it is more readable than
   load data substitutions */
INSERT INTO \`Order\` (
  SELECT id,
      STR_TO_DATE(TRIM('"' FROM datePlaced), '%Y-%m-%d %T'),
      STR_TO_DATE(NULLIF(TRIM('"' FROM dateShipped), 'Cancelled'),
        '%Y-%m-%d %T'),
      customer_id
    FROM OrderStaging
);

DROP TABLE OrderStaging;

/* Create a table for the raw data with types suited for them */
CREATE TABLE ProductStaging (
  id BIGINT UNSIGNED,
  name VARCHAR(100),
  price VARCHAR(30),
  quantity INTEGER
);
LOAD DATA LOCAL INFILE '/home/${UNAME}/products.csv'
  INTO TABLE ProductStaging
  FIELDS TERMINATED BY ','
    OPTIONALLY ENCLOSED BY '"'
  IGNORE 1 LINES
;
/* Use a select to transform the data as it is more readable
   than load data substitutions */
INSERT INTO Product (
  SELECT id,
      TRIM('"' FROM name),
      /* replace '$', ',', and '"' with empty string */
      CAST(REGEXP_REPLACE(price, '[$,"]', '') AS DECIMAL(6,2)),
      quantity
    FROM ProductStaging
);
INSERT INTO PriceHistory (oldPrice, newPrice, ts, product_id) (
  SELECT NULL,
      /* replace '$', ',', and '"' with empty string */
```

```
      CAST(REGEXP_REPLACE(price, '[$,"]', '') AS DECIMAL(6,2)),
      /* No data given so this is the 'new' official price as of NOW */
      NOW(),
      id
    FROM ProductStaging
);

DROP TABLE ProductStaging;

/* Create a table for the raw data with types suited for them */
CREATE TABLE OrderlineStaging (
  order_id BIGINT UNSIGNED,
  product_id BIGINT UNSIGNED
);
LOAD DATA LOCAL INFILE '/home/${UNAME}/orderlines.csv'
  INTO TABLE OrderlineStaging FIELDS TERMINATED BY ','
    OPTIONALLY ENCLOSED BY '"'
  IGNORE 1 LINES
;
/* Use a select to transform the data as it is more readable
   than load data substitutions */
INSERT INTO Orderline (
  SELECT order_id,
      product_id,
      COUNT(*)
    FROM OrderlineStaging
    GROUP BY order_id, product_id
);

DROP TABLE OrderlineStaging;
SUBEOF

sudo -u ${UNAME} cat > /home/${UNAME}/views.sql <<'SUBEOF'
SOURCE /home/${UNAME}/etl.sql

USE POS;

-- Handy view that will be used once ever
CREATE OR REPLACE VIEW v_ProductBuyers
  AS SELECT P.id productID, P.name productName,
      GROUP_CONCAT(
        DISTINCT CONCAT(C.id, ' ', C.firstName, ' ', C.lastName)
        ORDER BY C.id
        SEPARATOR ','
      ) customers
    FROM Customer C
    JOIN \`Order\` O ON O.customer_id = C.id
```

```
    JOIN Orderline OL ON OL.order_id = O.id
    -- Could benchmark this against an initial left join
    RIGHT JOIN Product P ON P.id = OL.product_id
    GROUP BY P.id, P.name
    ORDER BY P.id
;

-- Here it is, the single use of the view to create an MV table
CREATE TABLE IF NOT EXISTS mv_ProductBuyers ENGINE=InnoDB
  AS SELECT * FROM v_ProductBuyers
;

-- Boost performance by indexing the id
CREATE INDEX IF NOT EXISTS i_mv_ProductBuyers__productID
  ON mv_ProductBuyers (productID)
;

-- Prepare for code blocks
DELIMITER ||

-- Create procedure for shared update logic
CREATE OR REPLACE PROCEDURE p_update_customer_list(
  IN target_product_id BIGINT
)
BEGIN
  -- Refresh the list of customers that have ordered product
  UPDATE mv_ProductBuyers
    SET customers = (
      SELECT GROUP_CONCAT(
        DISTINCT CONCAT(C.id, ' ',
          C.firstName, ' ', C.lastName
        )
        ORDER BY C.id
        SEPARATOR ','
      )
      FROM \`Order\` O
      JOIN (
        SELECT DISTINCT OL.order_id id FROM Orderline OL
          WHERE OL.product_id = target_product_id
      ) PO ON PO.id = O.id
      -- At this point we have all product's orders
      JOIN Customer C ON C.id = O.customer_id
    )
    WHERE productID = target_product_id
  ;
END ||
```

```
-- Add triggers for both insert and delete to update mv_ProductBuyers
-- using p_update_customer_list
CREATE OR REPLACE TRIGGER t_mv_ProductBuyers__Orderline_insert
  AFTER INSERT ON Orderline
    FOR EACH ROW
    BEGIN
      CALL p_update_customer_list(NEW.product_id);
    END;
    ||

CREATE OR REPLACE TRIGGER t_mv_ProductBuyers__Orderline_delete
  AFTER DELETE ON Orderline
    FOR EACH ROW
    BEGIN
      CALL p_update_customer_list(OLD.product_id);
    END;
    ||

-- Only update price history when product price changes
CREATE OR REPLACE TRIGGER t_PriceHistory__Product_update
  AFTER UPDATE ON Product
    FOR EACH ROW
    BEGIN
      IF OLD.currentPrice != NEW.currentPrice THEN
        INSERT INTO PriceHistory(oldPrice, newPrice, ts, product_id)
          VALUES(OLD.currentPrice, NEW.currentPrice, NOW(), OLD.id)
        ;
      END IF;
    END;
    ||

-- Done with code blocks
DELIMITER ;

-- Verify creation success
SHOW CREATE VIEW v_ProductBuyers;
SHOW CREATE TABLE mv_ProductBuyers;
SELECT * FROM mv_ProductBuyers LIMIT 5;

SUBEOF

chown ${UNAME}:${UNAME} /home/${UNAME}/etl.sql
chmod 755 /home/${UNAME}/etl.sql
chown ${UNAME}:${UNAME} /home/${UNAME}/views.sql
chmod 755 /home/${UNAME}/views.sql

cat > /root/run_etl.sh <<'SUBEOF'
```

```
#!/bin/bash
# shebang so the shell knows this is a shell script

if [ ! -e "/home/${UNAME}/${TAMUUID}.zip" ]; then
  sudo -u ${UNAME} wget http://622.gomillion.org/data/${TAMUUID}.zip -O \
    /home/${UNAME}/${TAMUUID}.zip
  sudo -u ${UNAME} unzip /home/${UNAME}/${TAMUUID}.zip -d /home/${UNAME}
fi

# TODO aws secretsmanager workflow
mariadb <<SUBSUBEOF
CREATE DATABASE IF NOT EXISTS POS;
CREATE USER IF NOT EXISTS '${UNAME}'@'localhost' IDENTIFIED BY '${UPASSWORD}';
GRANT ALL PRIVILEGES ON POS.* TO '${UNAME}'@'localhost';
FLUSH PRIVILEGES;

SUBSUBEOF

# Allows creation of functions and triggers
mariadb -e "SET GLOBAL log_bin_trust_function_creators = 1;"
# Allows local data to be loaded
mariadb -e "SET GLOBAL local_infile = 1;"
sudo -u ${UNAME} cat /home/${UNAME}/views.sql \
  | sudo -u ${UNAME} mariadb -u ${UNAME} --password=${UPASSWORD}
# Return to safety
mariadb -e "SET GLOBAL log_bin_trust_function_creators = 0;"
mariadb -e "SET GLOBAL local_infile = 0;"

touch /root/7-etl-done
SUBEOF
chmod 700 /root/run_etl.sh

mkdir "${OUTFILE_DIR}"
chown mysql:mysql "${OUTFILE_DIR}"

cat > /root/export_json.sh <<'SUBEOF'
#!/bin/bash
if [ ! -d ${OUTFILE_DIR} ]; then
  mkdir "${OUTFILE_DIR}"
  chown mysql:mysql "${OUTFILE_DIR}"
fi

if [ ! -e "${PROD_OUT}" ]; then

mariadb <<'SUBSUBEOF'
USE POS;
```

```
SELECT JSON_OBJECT("ProductID", P.id,
    "ProductName", P.name,
    "ProductBuyers", JSON_ARRAYAGG(DISTINCT
      JSON_OBJECT("CustomerID", C.id,
        "CustomerName", CONCAT(C.firstName, " ", C.lastName)
      )
    )
  )
  FROM Product P
  LEFT JOIN Orderline OL ON OL.product_id = P.id
  JOIN \`Order\` O ON O.id = OL.order_id
  JOIN Customer C ON C.id = O.customer_id
  GROUP BY P.id, P.name
  ORDER BY P.id
  INTO OUTFILE "${PROD_OUT}"
;
SUBSUBEOF
jq . "$PROD_OUT" > "$PROD_OUT.pretty"
fi

if [ ! -e "${CUST_OUT}" ]; then
mariadb <<'SUBSUBEOF'
USE POS;
SELECT JSON_OBJECT(
    "CustomerID", C.id,
    "CustomerName", CONCAT(C.firstName, " ", C.lastName),
    "PrintedAddress1", CONCAT(address1,
      IF(C.address2 IS NOT NULL, CONCAT(" #", C.address2), "")
    ),
    "PrintedAddress2", CONCAT(Ci.city, ", ", Ci.state, " \ \ ", C.zip),
    "Orders", IFNULL(JSON_ARRAYAGG(OI.result), JSON_ARRAY())
  )
  FROM Customer C
  JOIN City Ci ON Ci.zip = C.zip
  -- Surprisingly a non-correlated subquery appears to be more efficient here
  -- I think the optimization engine is able to cache this query
  -- It saves an additional subquery but uses more memory
  -- If the data set gets larger it can be turned into a WITH/correlated
  LEFT JOIN (
    SELECT O.customer_id AS id,
      JSON_OBJECT(
        "OrderID", O.id,
        "OrderTotal", ROUND(SUM(P.currentPrice * OL.quantity), 2),
        "OrderDate", O.datePlaced,
        "ShippingDate", O.dateShipped,
        "Items", JSON_ARRAYAGG(
          JSON_OBJECT(
```

```
            "ProductID", P.id,
            "Quantity", OL.quantity,
            "ProductName", P.name
          )
        )
      ) AS result
    FROM \`Order\` O
    JOIN Orderline OL ON OL.order_id = O.id
    JOIN Product P ON P.id = OL.product_id
    GROUP BY O.id, O.datePlaced, O.dateShipped
    ORDER BY O.id
  ) OI ON OI.id = C.id
  GROUP BY C.id, C.firstName, C.lastName, C.address1, C.address2, OI.id
  ORDER BY C.id
  INTO OUTFILE "${CUST_OUT}"
;
SUBSUBEOF
jq . "$CUST_OUT" > "$CUST_OUT.pretty"
fi

if [ ! -e "$CUSTOM1_OUT" ]; then
mariadb <<'SUBSUBEOF'
# Business case 1
# There is a need to know which regions prefer which products
# to determine which stores to sell certain products at
# TODO examine historical price data to retrieve price at time of purchase
USE POS;
SELECT JSON_OBJECT(
    "ZIP", CAST(Ci.zip AS CHAR),
    "State", Ci.state,
    "City", Ci.city,
    "PopularProducts", COALESCE(JSON_ARRAYAGG(PP.result), JSON_ARRAY())
  )
  FROM City Ci
  JOIN (
    SELECT C.zip AS zip,
      JSON_OBJECT(
        "TotalPurchases", SUM(IFNULL(OL.quantity, 0)),
        "TotalSales", ROUND(
          SUM(IFNULL(OL.quantity, 0) * P.currentPrice), 2
        ),
        "ProductID", P.id,
        "ProductName", P.name
      ) AS result
    FROM Product P
    LEFT JOIN Orderline OL ON OL.product_id = P.id
    JOIN \`Order\` O ON O.id = OL.order_id
```

```
    JOIN Customer C ON C.id = O.customer_id
    GROUP BY C.zip, P.id, P.name
    ORDER BY P.id
  ) PP ON PP.zip = Ci.zip
  GROUP BY Ci.zip, Ci.state, Ci.city
  INTO OUTFILE "$CUSTOM1_OUT"
;
SUBSUBEOF
jq . "$CUSTOM1_OUT" > "$CUSTOM1_OUT.pretty"
fi

if [ ! -e "$CUSTOM2_OUT" ]; then
mariadb <<'SUBSUBEOF'
# Business case 2
# There is a need to know what times of year to keep certain items
# in higher regional stock so that shipping times are not effected
# TODO examine historical price data to retrieve price at time of purchase
USE POS;
SELECT JSON_OBJECT(
    "OrderMonth", PP.orderMonth,
    "State", Ci.state,
    "PopularProducts", COALESCE(JSON_ARRAYAGG(PP.result), JSON_ARRAY())
  )
  FROM City Ci
  JOIN (
    SELECT C.zip AS zip,
      MONTH(O.datePlaced) AS orderMonth,
      JSON_OBJECT(
        "TotalPurchases", SUM(OL.quantity),
        "TotalSales", ROUND(
          SUM(OL.quantity * P.currentPrice), 2
        ),
        "ProductID", P.id,
        "ProductName", P.name
      ) AS result
    FROM Product P
    LEFT JOIN Orderline OL ON OL.product_id = P.id
    JOIN \`Order\` O ON O.id = OL.order_id
    JOIN Customer C ON C.id = O.customer_id
    GROUP BY C.zip, P.id, P.name, MONTH(O.datePlaced)
  ) PP ON PP.zip = Ci.zip
  GROUP BY Ci.state, PP.orderMonth
  INTO OUTFILE "$CUSTOM2_OUT"
;
SUBSUBEOF
jq . "$CUSTOM2_OUT" > "$CUSTOM2_OUT.pretty"
fi
```

```
SUBEOF
chmod 700 /root/export_json.sh

touch /root/8-export_json

# Make mongodb slightly more secure
# TODO aws secretsmanager workflow
mongosh admin --eval 'db.createUser({
  user: "root",
  pwd: "$ADMINPASS",
  roles: [ { role: "root", db: "admin" } ]
}) ;
exit()'

# Actually this uses too many characters right now to convert to using auth
#sed -i "s/^#*security:/security:/" \
#/etc/mongod.conf
#sed -i "/security:/a\
#\ \ authorization: enabled" /etc/mongod.conf

mongosh --quiet <<'SUBEOF'
use POS
db.createUser({
  user: "$UNAME",
  pwd: "$UPASSWORD",
  roles: [{ role: "readWrite", db: "POS" }]
})
SUBEOF

cat > /root/sync.sh <<'SUBEOF'
#!/bin/bash
SHELL=/bin/bash
PATH=/usr/local/sbin:/usr/local/bin:/sbin:/bin:/usr/sbin:/usr/bin
# CLEAN
if [ ! -d ${OUTFILE_DIR} ]; then
  mkdir ${OUTFILE_DIR}
  chown mysql:mysql ${OUTFILE_DIR}
fi

if [ -e "${PROD_OUT}" ]; then
  rm "$PROD_OUT"
fi

if [ -e "${CUST_OUT}" ]; then
  rm "$CUST_OUT"
fi
```

```
if [ -e "$CUSTOM1_OUT" ]; then
  rm "$CUSTOM1_OUT"
fi

if [ -e "$CUSTOM2_OUT" ]; then
  rm "$CUSTOM2_OUT"
fi

# EXTRACT
timeout 300 /root/export_json.sh || { echo "Export timed out"; exit 1; }

# LOAD
JSON_NAMES=( "prod" "cust" "custom1" "custom2" )
for coll in "\${JSON_NAMES[@]}"; do
  # Import to temp collections
  mongoimport --db POS --collection \${coll}_tmp --drop \
    --file="${OUTFILE_DIR}/\${coll}.json"
  # Rename happens instantly(atomically) so no worries about lock conflicts
  # There is a bug that causes mongosh not to terminate with --eval
  # so the timeout wrapper and exit() will hopefully prevent that and
  # make sure the system doesn't hang and gobble CPU.
  # I spent the mojority of my time on this part of the assignment because
  # I could not figure out why mongosh kept hanging. Eventually Claude was
  # able to tell me why
  timeout 30 \
    mongosh --eval "db.adminCommand({ renameCollection: 'POS.\${coll}_tmp',
      to: 'POS.\${coll}', dropTarget: true }) ;
      exit()" || { echo "mongosh timed out"; exit 1 ; }
done

SUBEOF

chmod 700 /root/sync.sh

echo "$CRON_JOB" | crontab -
#(crontab -l | grep -v -F "$CRON_CMD"; echo "$CRON_JOB") | crontab -

touch /root/9-sync_scheduled

cat > /root/run_queries.sh <<'SUBEOF'
#!/bin/bash
# The Texas Campaign
# Write a query against the Customers collection that filters by State.
mongosh POS --quiet --eval 'printjson(db.cust.find({ PrintedAddress2:
  { \$regex: /.* TX   .*/ }}).toArray()) ;
  exit()' > /root/q1.json
```

```
# The VIP Customer
# Write a query to find the customer who has spent the most money or placed
# the most orders
mongosh POS --quiet --eval 'printjson(db.cust.aggregate([
  { \$addFields: { totalOrderCost: { \$sum: "\$Orders.OrderTotal" } } },
  { \$sort: { totalOrderCost: -1 } },
  { \$group: { _id: null, maxCost: { \$first: "\$totalOrderCost" },
        customers: { \$push: "\$\$ROOT" } } },
  { \$project: {
      VIPs: {
        \$filter: {
          input: "\$customers",
          cond: { \$eq: ["\$\$this.totalOrderCost", "\$maxCost"] }
        }
      }
  }}
])
);
  exit()' > /root/q2.json

# Targeted Recall Notice
# Pick a specific product ID or Name. Write a query against the Products
# collection to return the array of buyers for that specific item
mongosh POS --quiet --eval 'printjson(db.prod.find({ ProductID: 1 },
    { ProductName: 1, ProductBuyers: 1 }).toArray());
  exit()' > /root/q3.json

# Fraud Detection
# Write a creative query against the Customers collection to look for
# anomalies (e.g., finding a customer who bought 50 of the same expensive
# item, or searching for suspiciously high order totals)
mongosh POS --quiet --eval 'printjson(db.cust.find({ \$or: [
    { "Orders.OrderTotal": { \$gt: 20000 }},
    { "Orders.Items.Quantity": { \$gt: 10 }}]}).toArray());
  exit()' > /root/q4.json

# The Architect's Custom Cases
# Write one powerful query for each of your custom collections that
# demonstrates why you built that aggregate. Show the business value

# The College Station outlet wants to know what products are popular in Texas
# so they can expand their product line
mongosh POS --quiet --eval 'printjson(db.custom1.aggregate([
  { \$match: { State: "TX" } },
  { \$unwind: "\$PopularProducts" },
  { \$group: { _id: "\$PopularProducts.ProductID",
```

```
      ProductName: { \$first: "\$PopularProducts.ProductName" },
      TotalStatePurchases: { \$sum: "\$PopularProducts.TotalPurchases" } } },
  { \$sort: { TotalStatePurchases: -1 } },
  { \$limit: 20 }
])
);
  exit()' > /root/q5.json

# The central Texas warehouse located in Austin needs to know which items
# to stock so they can stay in stock during tax refund season
mongosh POS --quiet --eval 'printjson(db.custom2.aggregate([
  { \$match: { State: "TX", OrderMonth: 5 } },
  { \$unwind: "\$PopularProducts" },
  { \$group: { _id: "\$PopularProducts.ProductID",
      ProductName: { \$first: "$PopularProducts.ProductName" },
      TotalStatePurchases: { \$sum: "\$PopularProducts.TotalPurchases" } } },
  { \$sort: { TotalStatePurchases: -1 } }
])
);
  exit()' > /root/q6.json

SUBEOF
chmod 700 /root/run_queries.sh

touch /root/10-startup-done

rm /root/0-script-failed

echo "SUCCESS"
EOF

echo "Script length: ${#STARTUP} characters"
# Save space by removing bash comments
STARTUP=$(echo "$STARTUP" | \
  sed '1!{s/^[[:blank:]]*#[^!].*//}; 1!{/^[[:blank:]]*$/d}')
echo "Script length: ${#STARTUP} characters"
# Save space by removing SQL comments
STARTUP=$(echo "$STARTUP" | sed 's/^[[:blank:]]*--.*//; /^[[:blank:]]*$/d')
echo "Script length: ${#STARTUP} characters"
# Save space by squashing whitespace
STARTUP=$(echo "$STARTUP" | sed 's/  */ /g; s/^ //')
echo "Script length: ${#STARTUP} characters"

# $1=NAME $2=INSTANCEID
run_instance() {
local -n IID=$2
  IID="$( aws ec2 run-instances \
```

```
    --tag-specifications "ResourceType=instance,Tags=[{Key=Name,Value=$1}]" \
    --image-id "$IMAGEID" \
    --count 1 \
    --instance-type "$INSTTYPE" \
    --key-name "$KEYNAME" \
    --security-group-ids "$SGID" \
    --query "Instances[0].InstanceId" \
    --iam-instance-profile Name="$IAMPROFILE" \
    --user-data "${STARTUP}" \
    --output text )"
}

# BEGIN AWS CLI script
{
# Check for security group and create if it doesn't exist
SGID=$(aws ec2 describe-security-groups \
  --filters Name=group-name,Values="$SGNAME" \
  --query "$SGIDPARSE" \
  --output text)
if [ $? -eq 0 ] && [ ${#SGID} -gt 7 ];
then
  echo "Security Group '$SGNAME' Exists"
elif [ $? -eq 0 ] && [ ${#SGID} -lt 8 ];
then
  echo "Creating Security Group '$SGNAME'"
  # Add security group and retrieve new Group ID
  SGID=$(aws ec2 create-security-group \
    --group-name $SGNAME \
    --description "$SGDESC" \
    --query "GroupId" \
    --output text)
  # Add SSH port rule
  aws ec2 authorize-security-group-ingress \
    --group-id "$SGID" \
    --protocol tcp \
    --port 22 \
    --cidr "$MYIP"
  # Add MariaDB and Galera ports
  for i in "${PORTS[@]}";
  do
    PAIR=( $i )
    aws ec2 authorize-security-group-ingress \
      --group-id "$SGID" \
      --ip-permissions \
        "$(ip_permissions_str $SGID ${PAIR[0]} ${PAIR[1]} $MYACCT)"
  done
fi
```

```
# Start Instance A and retrieve AWS instance ID
# These need to be in quotes because of the startup script included
run_instance "A" INSTANCEA
echo "Instance created with ID: $INSTANCEA"
echo "aws ec2 terminate-instances --instance-ids $INSTANCEA"

# Give a little time for everything to start smoothly
sleep 1

# Wait for instance A to boot
while [ $(aws ec2 describe-instances \
  --filters "Name=instance-id,Values=$INSTANCEA" \
  --query "Reservations[].Instances[].State.Name" \
  --output text ) != "running" ];
do
  echo "Wait for instance A to finish boot..."
  sleep $POLL_INTERVAL
done

# Instance A should have a private IP now
INSTANCEAIP=$(aws ec2 describe-instances \
  --filters "Name=instance-id,Values=$INSTANCEA" \
  --query "Reservations[].Instances[].PrivateIpAddress" \
  --output text | sed "s/-/\./g" )

echo "Instance A has started with local IP $INSTANCEAIP"

# Wait for instance A to be ready for commands
while [ "$(aws ssm get-connection-status \
        --target $INSTANCEA \
        --query 'Status' \
        --output 'text')" != "connected" ];
do
  echo "Wait for instance A to finish initializing..."
  sleep $POLL_INTERVAL
done
echo "Instance A is ready for commands"

# It sometimes takes a few seconds to actually be ready after it says it is
sleep $POLL_INTERVAL

comm_send_wait "$INSTANCEA" "cloud-init status --wait" \
  "Wait for instance A to finish setup..." COMMID

# Retreive final command response code because this tells if it worked
if [[ $? -ne 0 || $(aws ssm get-command-invocation \
```

```
  --command-id "$COMMID" \
  --instance-id "$INSTANCEA" \
  --query "ResponseCode" \
  --output text ) -ne 0 ]]; then
  $("${UNPASTE}")
fi
echo "A has finished setup"

# Run etl on instance A
comm_send_wait "$INSTANCEA" "/root/run_etl.sh" \
  "Wait for instance A to finish etl..." COMMID

# Retreive final command response code because this tells if it worked
if [[ $? -ne 0 || $(aws ssm get-command-invocation \
  --command-id "$COMMID" \
  --instance-id "$INSTANCEA" \
  --query "ResponseCode" \
  --output text ) -ne 0 ]]; then
  $("${UNPASTE}")
fi
echo "Instance A has finished etl"

# Run json export on instance A

comm_send_wait "$INSTANCEA" "/root/export_json.sh" \
  "Wait for instance A to finish json export..." COMMID

# Retreive final command response code because this tells if it worked
if [[ $? -ne 0 || $(aws ssm get-command-invocation \
  --command-id "$COMMID" \
  --instance-id "$INSTANCEA" \
  --query "ResponseCode" \
  --output text ) -ne 0 ]]; then
  $("${UNPASTE}")
fi
echo "Instance A has finished json export"

sleep $POLL_INTERVAL

# Make sure a sync has been run
comm_send_wait "$INSTANCEA" "flock -n /tmp/mysync.lock /root/sync.sh" \
  "Wait for instance A to finish first sync..." COMMID
echo "Instance A has finished sync"

# Run MongoDB queries
comm_send_wait "$INSTANCEA" "/root/run_queries.sh" \
  "Wait for instance A to finish queries..." COMMID
```

```
echo "Instance A has finished running queries"

echo "SUCCESS"

# TESTS FOR RUNNING
echo "crontab -l"
echo "flock -n /tmp/mysync.lock mongosh"
echo "use POS; \ndb.cust.find({CustomerID: 1})"
echo "USE POS; UPDATE Product SET currentPrice = 9999 WHERE id = 449;"
echo "flock -n /tmp/mysync.lock /root/sync.sh"
echo "mongosh POS --quiet --eval 'printjson(db.cust.find({ PrintedAddress2: \
  { \$regex: /.* TX   .*/ }}).toArray())' > /root/q1.json && \
  less '/root/q1.json'"
echo "mongosh POS --quiet --eval 'printjson(db.cust.aggregate([ \
  { \$addFields: { totalOrderCost: { \$sum: \"\$Orders.OrderTotal\" } } }, \
  { \$sort: { totalOrderCost: -1 } }, \
  { \$group: { _id: null, maxCost: { \$first: \"\$totalOrderCost\" }, \
        customers: { \$push: \"\$\$ROOT\" } } }, \
  { \$project: { \
      VIPs: { \
        \$filter: { \
          input: \"\$customers\", \
          cond: { \$eq: [\"\$\$this.totalOrderCost\", \"\$maxCost\"] } \
        } \
      } \
  }} \
]) \
)' > root/q2.json && \
  less '/root/q2.json'"
echo "mongosh POS --quiet --eval 'printjson(db.prod.find({ ProductID: 1 }, \
    { ProductName: 1, ProductBuyers: 1 }).toArray())' > /root/q3.json && \
  less '/root/q3.json'"
echo "mongosh POS --quiet --eval 'printjson(db.cust.find({ \$or: [ \
    { \"Orders.OrderTotal\": { \$gt: 20000 }}, \
    { \"Orders.Items.Quantity\": { \$gt: 10 }}]}).toArray())' > /root/q4.json \
    && less '/root/q4.json'"
echo "mongosh POS --quiet --eval 'printjson(db.custom1.aggregate([ \
  { \$match: { State: \"TX\" } }, \
  { \$unwind: \"\$PopularProducts\" }, \
  { \$group: { _id: \"\$PopularProducts.ProductID\", \
    ProductName: { \$first: \"\$PopularProducts.ProductName\" }, \
    TotalStatePurchases: { \$sum: \"\$PopularProducts.TotalPurchases\" } } }, \
  { \$sort: { TotalStatePurchases: -1 } }, \
  { \$limit: 20 } \
]) \
)' > /root/q5.json && \
  less '/root/q5.json'"
```

```
echo "mongosh POS --quiet --eval 'printjson(db.custom2.aggregate([ \
  { \$match: { State: \"TX\", OrderMonth: 5 } }, \
  { \$unwind: \"\$PopularProducts\" }, \
  { \$group: { _id: \"\$PopularProducts.ProductID\", \
    ProductName: { \$first: \"$PopularProducts.ProductName\" }, \
    TotalStatePurchases: { \$sum: \"\$PopularProducts.TotalPurchases\" } } }, \
  { \$sort: { TotalStatePurchases: -1 } } \
]) \
)' > /root/q6.json && \
  less '/root/q6.json'"


# Connect to
echo "Instance A public IP:   $(aws ec2 describe-instances \
  --filters "Name=instance-id,Values=$INSTANCEA" \
  --query "Reservations[].Instances[].PublicIpAddress" \
  --output text)"

# WHEN DONE, DELETE INSTANCES WITH
echo "aws ec2 terminate-instances --instance-ids $INSTANCEA"

}
```

4. Query Cheat Sheet: Paste the plain text of the 6 MongoDB queries you wrote.

```
1. db.cust.find({ PrintedAddress2: {$regex: /.* TX   .*/ }})
2. db.cust.aggregate([
      { $addFields: { totalOrderCost: { $sum: "$Orders.OrderTotal" } } },
      { $sort: { totalOrderCost: -1 } },
      { $group: { _id: null, maxCost: { $first: "$totalOrderCost" },
         customers: { $push: "$$ROOT" } } },
      { $project: {
         VIPs: {
            $filter: {
               input: "$customers",
               cond: { $eq: ["$$this.totalOrderCost", "$maxCost"] }
            }
         }
      }}
   ])
3. db.prod.find({ ProductID: 1 }, { ProductName: 1, ProductBuyers: 1 })
4. db.cust.find({ $or: [ { "Orders.OrderTotal": { $gt: 20000 }},
      { "Orders.Items.Quantity": { $gt: 10 }}]})
5. db.custom1.aggregate([
      { $match: { State: "TX" } },
      { $unwind: "$PopularProducts" },
      { $group: { _id: "$PopularProducts.ProductID",
         ProductName: { $first: "$PopularProducts.ProductName" },
         TotalStatePurchases: { $sum: "$PopularProducts.TotalPurchases" } } },
      { $sort: { TotalStatePurchases: -1 } },
      { $limit: 20 }
   ])
6. db.custom2.aggregate([
      { $match: { State: "TX", OrderMonth: 5 } },
      { $unwind: "$PopularProducts" },
      { $group: { _id: "$PopularProducts.ProductID",
         ProductName: { $first: "$PopularProducts.ProductName" },
         TotalStatePurchases: { $sum: "$PopularProducts.TotalPurchases" } } },
      { $sort: { TotalStatePurchases: -1 } }
   ])
```

4. Video Outline (with Timestamps):
- Time 0:00 - Intro & Pipeline Infrastructure (sync.sh and crontab).
- Time 2:00 - Pipeline Demo (MariaDB Insert -> sync.sh -> MongoDB Verification).
- Time 3:00 - Query 1 Execution: Texas Customers.
- Time 4:00 - Query 2 Execution: VIP Customer.
- Time 6:00 - Query 3 Execution: Targeted Recall.
- Time 6:40 - Query 4 Execution: Fraud Detection.
- Time 8:20 - Queries 5 & 6 Execution: Custom Business Cases.
- Time 12:00 - AI Disclosure & Reflection (Polyglot Persistence & SQL vs Mongo).